

DSA612S

Assignment 1

Presentation Runbook

Distributed Systems and Applications

A complete step-by-step guide to demonstrating this project: setting up VS Code, running the REST service and its client, opening and explaining the web dashboard, running the gRPC server and client, executing the tests, and handling anything that goes wrong live.

Question 1 — Distributed Library and Resource Management System (REST)

Question 2 — Rental Accommodation System (gRPC)

Group 22

Matatias Nghihangwa	225156237
Eliaser Angula (leader)	225053241
Pandera Katjipuka	225123851
Monika Shalauda	222075449
Risto Sakeus	225042428
Alanray Miller	223003018
Kavara Edward	225017288
Haufiku Frans	222127147

Verified on Windows 11 with XAMPP and Ballerina 2201.13.5 (Swan Lake Update 13).

A step-by-step guide for demonstrating and defending this project, from opening VS Code to answering the last question.

Everything in this guide was executed and verified on the project machine (Windows 11, XAMPP, Ballerina 2201.13.5). The commands are the exact ones that work.

Contents

Part	What it covers
0	Before the presentation — setup and checks
1	VS Code layout for the demo
2	Question 1 — REST API and command-line client
3	The bonus web dashboard
4	Question 2 — gRPC server and client
5	Running the automated tests
6	Troubleshooting during the live demo
7	Questions the lecturer is likely to ask
8	Timing plan and speaking roles
9	One-page command cheat sheet

Part 0 — Before the presentation

Do this the night before **and** again fifteen minutes before you present.

0.1 Confirm Ballerina is installed

Open PowerShell and run:

```
bal version
```

Expected:

```
Ballerina 2201.13.5 (Swan Lake Update 13)
Language specification 2024R1
Update Tool 1.5.1
```

If you get *"bal is not recognised"*, Ballerina is installed but not on your PATH. It lives at `C:\Program Files\Ballerina`. Fix it once:

1. Press **Win** and type **environment variables**, open *Edit the system environment variables*.
2. Click **Environment Variables...**
3. Under *User variables*, select **Path**, click **Edit**, then **New**.
4. Paste `C:\Program Files\Ballerina\bin`
5. Click OK on all dialogs.
6. **Close and reopen VS Code** — PATH changes only apply to new terminals.

Verify with `bal version` again before you rely on it.

0.2 The port 8080 trap — read this

XAMPP's Apache occupies port **8080** on this machine. The REST service defaults to 8080, so if Apache is running the service cannot start and you will see:

```
error: failed to start server connector '0.0.0.0:8080': Address already in use: bind
```

You have two options. **Pick one and rehearse it:**

- **Option A (recommended, nothing to turn off):** run the REST service on port **8081** by passing a config value. Every command in this guide already does this.
- **Option B:** stop Apache in the XAMPP Control Panel and use the default 8080. Only do this if you are sure nothing else needs XAMPP.

Ports used in this guide:

Service	Port
Question 1 REST API	8081
Question 2 gRPC server	9090
Web dashboard (static files)	5500
XAMPP Apache — leave alone	8080

0.3 Pre-build everything

Compiling takes 30–60 seconds per package. **Never do this in front of the audience.** Build everything beforehand so every `bal run` starts instantly.

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment"  
.\scripts\run.ps1 -Target build
```

This builds all four packages. Expected output ends with four `target\bin\*.jar` lines and no `ERROR` lines.

If you prefer to do it manually:

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question1\service"  
bal build  
cd ..\client  
bal build  
cd ../../Question2\server  
bal build  
cd ..\client  
bal build
```

0.4 Rehearsal checklist

Tick every box before you present:

- ☐ `bal version` prints 2201.13.5
- ☐ All four packages build with no errors
- ☐ REST server starts on 8081 and `/health` returns JSON
- ☐ REST client connects and lists 6 assets
- ☐ Web dashboard shows **API online** and 6 assets

- [] gRPC server starts on 9090 and prints its 8 RPCs
- [] gRPC client connects and completes a booking
- [] `bal test` passes in both `Question1/service` and `Question2/server`
- [] Laptop set to **never sleep** during the presentation
- [] Notifications silenced (Windows Focus Assist on)

0.5 Important habit — restart servers after rebuilding

If you run `bal build` while a server is still running, the running server keeps using the old file and will start throwing strange errors (timeouts, 500s, class-loading failures).

Rule: stop the server (`Ctrl+C`), rebuild, then start it again.

Part 1 — VS Code layout for the demo

1.1 Open the project

1. Launch **VS Code**.
2. **File** → **Open Folder...**
3. Select `C:\xampp\htdocs\DISTRIBUTIVE_SYSTEM\DSA612S-Assignment`
4. Click **Select Folder**.

The Explorer on the left should show `Question1`, `Question2`, `docs`, `scripts`, and `README.md`.

*Install the **Ballerina** extension (Extensions panel → search "Ballerina" → Install). It gives syntax highlighting, which makes the code far more readable on a projector.*

1.2 Make the screen projector-friendly

Do this before anyone is watching:

- Zoom the whole UI: `Ctrl + =` two or three times (`Ctrl + 0` resets).
- Hide the sidebar when showing terminals: `Ctrl + B` toggles it.
- Switch to a light theme if the room is bright:

`Ctrl + K` then `Ctrl + T`.

1.3 Open the terminal panel and split it

Press `Ctrl + `` (backtick) to open the terminal.

Click the `+` icon (or `Ctrl + Shift + ``) to add terminals, and the **split** icon to place them side by side. You need **four**:

Terminal	Purpose	Working directory
1	REST server (stays running)	<code>Question1\service</code>
2	REST client (interactive)	<code>Question1\client</code>
3	gRPC server (stays running)	<code>Question2\server</code>
4	gRPC client (interactive)	<code>Question2\client</code>

Rename each one so you don't get lost mid-demo: right-click the terminal tab → **Rename** → type `REST server`, `REST client`, etc.

A fifth terminal for the web dashboard is optional — see Part 3.

1.4 Files to have open in tabs

Open these ahead of time so you can switch to them instantly instead of hunting through the Explorer:

Question 1

- `Question1/service/models.bal` — the data model
- `Question1/service/database.bal` — the in-memory store
- `Question1/service/services.bal` — business rules
- `Question1/service/main.bal` — the REST endpoints

Question 2

- `Question2/proto/rental.proto` — the contract
 - `Question2/server/service bal` — the RPC implementations
 - `Question2/client/client bal` — the client
-

Part 2 — Question 1: REST API (50 marks)

2.1 Opening line (about 30 seconds)

"Question 1 is a distributed library and resource management system for the Ministry of Higher Education. It's a Ballerina REST service that tracks books, electronic equipment and physical spaces across multiple institutions and campuses. Every asset is identified by a unique `assetTag`. We also built a command-line client and a bonus web dashboard, both of which talk to the service over HTTP."

2.2 Walk the code — the order that tells a story

Spend about two minutes here. Go **model** → **storage** → **rules** → **endpoints**; that order explains itself.

`models.bal` — the vocabulary

Point at the enums at the top:

"These are our status values — `AVAILABLE`, `LOANED_OUT`, `OCCUPIED`, `UNDER_MAINTENANCE`, `DISPOSED`. Using an enum instead of a plain string means Ballerina rejects an invalid status at the boundary, before it ever reaches our logic."

Then scroll to the `Asset` record:

"This mirrors the payload in the assignment brief exactly — `assetTag`, `name`, `description`, `institution`, `site`, `status`, `dateAcquired`, plus nested arrays of components, schedules and work orders."

`database.bal` — the database requirement (5 marks)

Point at the very first line:

```
isolated table<Asset> key(assetTag) assetTable = table [];
```

"The brief asked for a Map or a table keyed on the unique identifier. This is a Ballerina `table` keyed on `assetTag`, so lookups are direct and duplicate tags are impossible. `isolated` plus the `lock` blocks below are what make it safe when several requests arrive at once."

`services.bal` — the business rules

Show `loanAsset` and point out `claimAssetForLoan`:

"Checking that an asset is available and then marking it as loaned has to happen as one atomic step. If we checked first and wrote afterwards, two students clicking at the same moment could both be issued the same laptop. The check and the write happen inside a single lock, so exactly one borrower wins. We have a test that fires 32 simultaneous loan requests and asserts that only one succeeds."

`main.bal` — the endpoints

Scroll through the `resource function` list:

"Twenty-five endpoints: full CRUD on assets, filtering by institution and by site, the overdue-maintenance report, institution management, component management, schedule management and work orders."

2.3 Start the REST server

Go to **Terminal 1 (REST server)**:

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question1\service"
bal run -- -CservicePort=8081
```

Expected output:

```
=====
DSA612S - Library and Resource Management System (REST)
Ministry of Higher Education, Training and Innovations
=====
Listening on : http://localhost:8081
Seeded assets: 6
Health check : http://localhost:8081/health
Press Ctrl+C to stop the server.
=====
```

Say:

"The service is up on port 8081 with six seeded assets from three different institutions. We're using 8081 because XAMPP already owns 8080 on this machine — the port is configurable, which is itself good practice for a distributed system."

2.4 Prove it responds

In any spare terminal:

```
curl http://localhost:8081/health
```

Expected:

```
{"status":"UP", "service":"library-resource-management", "version":"1.0.0",
  "timestamp":"...", "assets":6}
```

2.5 Run the command-line client

Go to **Terminal 2 (REST client)**:

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question1\client"
bal run -- -CapiUrl=http://localhost:8081
```

You should see:

```
Connected to http://localhost:8081 - 6 asset(s) in the store.
```

Say:

"This is a completely separate Ballerina package. It shares no memory with the server — every menu option below is a real HTTP request across a process boundary. That's the inter-process communication the assignment asked for."

The menu

1. View Assets (global view)
2. Search Asset (by tag or free text)
3. Loan Asset (issue / book a space)
4. Return Asset (hand back / release)
5. View Institution Assets (filter by institution)
6. View Campus Assets (filter by site)
7. View Overdue Maintenance (staff dashboard)
8. Manage Schedules (add / modify / remove)
9. Create Work Order (fault reporting)
10. Exit

The demo sequence — and what to say at each step

Run these **in this order**. It builds a narrative instead of jumping about.

Step 1 — Press **1** (Global view)

Six assets appear across NUST, UNAM and IUM.

"This is the ministry-wide view — every asset at every institution in one place. That's the View all assets requirement."

Step 2 — Press **5** (Filter by institution)

Pick **2** for Namibia University of Science and Technology.

"Now we're filtering to a single institution. Notice the client asks the server for the list of institutions first, then requests only that institution's assets — the filtering happens server-side, not in the client."

Step 3 — Press **6** (Filter by campus/site)

Pick any listed site.

"Same again, but by campus. The ministry has many campuses per institution, so both levels of filtering are needed."

Step 4 — Press **3** (Loan an asset)

- The client first shows only **AVAILABLE** assets.
- Asset tag: **NUST-LIB-LAP-014**
- Borrower: any student number, e.g. **221012345**
- Return date: press **Enter** to accept the default 14 days
- Room booking? **n**

*"The status has changed to **LOANED_OUT** and a loan record was created with a due date. Note the client only offered assets that were actually available — it never lets you attempt an invalid loan."*

Step 5 — Press **3** again and try the same tag

"And here's the error handling. The asset is no longer available, so the server rejects it with HTTP 409 Conflict and a readable message. The client turns that into a clean error line instead of crashing."

(This is a strong moment — it demonstrates the error-handling marks live.)

Step 6 — Press 4 (Return the asset)

- Asset tag: `NUST-LIB-LAP-014`
- Damaged? `y`
- Describe the damage: `Cracked screen`

"Because we flagged it as damaged, the server did two things at once: it moved the asset to `UNDER_MAINTENANCE` instead of back to `AVAILABLE`, and it automatically raised a work order. The business rule lives on the server, so every client gets the same behaviour."

Step 7 — Press 7 (Overdue dashboard)

Press **Enter** to see all institutions.

"This is the staff overdue view. The server walks every schedule on every asset, finds the ones whose due date has passed, and returns them sorted by how late they are. Four schedules are currently overdue in the seed data."

Step 8 — Press 8 (Manage schedules)

- Choose `1` to add
- Asset tag: `NUST-LIB-3DP-001`
- Type: `1` (MAINTENANCE)
- Due date: `2027-03-01`
- Description: `Quarterly calibration`
- Schedule id: press **Enter** to let the server generate one

"Schedules cover both servicing and room bookings. Option 8 also lets you modify or remove a schedule, which covers the add/remove schedule marks."

Step 9 — Press 9 (Create a work order)

- Asset tag: `NUST-LIB-3DP-001`
- Fault: `Nozzle heat-bed failure`
- Status: `1` (OPEN)
- Task: `Check thermal sensor connectivity`, then blank line to finish

"Work orders carry sub-tasks, exactly as in the sample payload in the brief. Opening one moves the asset to `UNDER_MAINTENANCE` automatically."

Step 10 — Press 10 to exit (or leave it running for questions).

2.6 Tie it back to the mark sheet

Say this explicitly — it helps the marker:

"That covered all seven line items: create and manage resources, view all assets, filter by institution and site, item status and booking schedules, manage institutions, manage schedules, and error handling."

Part 3 — The bonus web dashboard

This is worth up to **10 bonus marks**. Do not skip it.

3.1 Start a web server for the files

The dashboard is plain HTML, CSS and JavaScript. It must be served over HTTP (not opened as a file) so the browser will allow it to call the API.

Open a **fifth terminal** and run:

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment"
.\scripts\run.ps1 -Target web
```

Or manually:

```
python -m http.server 5500 --directory "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question1\web"
```

Leave it running.

3.2 Open it in the browser

Go to exactly this address (the **apiBase** part points it at port 8081):

```
http://localhost:5500/?apiBase=http://localhost:8081
```

*The **REST** server from Part 2 must still be running, or the dashboard will show "Connecting..." and no data.*

3.3 Explain the screen, region by region

Walk the page top to bottom. This is the script:

The header

*"Top left is the system title. Top right is a live status badge — it says **API online**, which means the page successfully called the **/health** endpoint. Next to it is a refresh button, a dark/light theme toggle, and **New Asset** for creating a resource straight from the browser."*

Click the **theme toggle** once — it's a nice touch and takes a second.

The five statistic cards

*"These five cards are computed live from the API, not hard-coded: **Total assets** across all institutions, **Available** and ready to issue, **Out / Occupied** currently on loan, **Under maintenance** withdrawn from service, and **Overdue schedules** needing attention now. Watch the numbers change when we loan something."*

The four tabs

*"**Assets** is the full catalogue. **Overdue Maintenance** is the staff dashboard — the red badge shows how many are late. **Work Orders** lists every open repair job with its sub-tasks. **Loan History** is the audit trail of who borrowed what and when."*

Click through all four tabs as you say this.

The search and filter row

"Free-text search across tag, name, description and campus, then three dropdowns — filter by institution, by campus or site, and by status. These map directly onto the API's query parameters, so the filtering is done by the server."

Demonstrate: type `printer` in the search box, then clear it. Then pick an institution from the dropdown, then click **Clear**.

The table

"The columns are asset tag, name, institution, campus, status, the number of schedules, the number of work orders, and the row actions."

The row actions — do one live

Point at the buttons on any row: **View**, **Loan**, **Return**, **Schedule**, **W/O** (raise a work order) and **Delete**. Buttons that don't apply are greyed out — an asset already on loan cannot be loaned again, so its **Loan** button is disabled and **Return** is enabled.

Click **View** on `NUST-LIB-3DP-001`:

"The detail modal shows the full asset — its components, its schedules and its work orders with every sub-task. This is the same JSON payload from the assignment brief, just rendered."

Close it, then click **Loan** on `NUST-LIB-LAP-014`. Enter a borrower such as `221012345`, leave the return date at its default (it pre-fills 14 days ahead), and click **Issue asset**.

A toast confirms *"NUST-LIB-LAP-014 is now out with 221012345"*, the row status flips to **Loaned out**, and its **Loan** button greys out.

Now scroll to the top of the page:

*"Look at the cards — **Available** dropped from 4 to 3, and **Out / Occupied** went from 0 to 1 active loan. Nothing was hard-coded; the page re-read those figures from the API. The browser and the command-line client are both talking to the same Ballerina service, so they always agree."*

Strong closing move: switch back to the command-line client (Terminal 2), press `1`, and show the same change there.

"Same data, two completely different clients, one service. That is the distributed part of the assignment."

Part 4 — Question 2: gRPC (50 marks)

4.1 Opening line (about 30 seconds)

"Question 2 is a rental accommodation platform for the Ministry of Tourism, built on gRPC instead of REST. There are two roles — hosts, who manage property listings, and guests, who search and book. We use all three call styles the brief asked for: simple unary calls, client-side streaming and server-side streaming."

4.2 Show the contract first — `rental.proto` (15 marks)

Open `Question2/proto/rental.proto` and scroll to the bottom:

```
service RentalService {
  rpc add_property (AddPropertyRequest) returns (AddPropertyResponse);
  rpc update_property (UpdatePropertyRequest) returns (UpdatePropertyResponse);
  rpc remove_property (RemovePropertyRequest) returns (RemovePropertyResponse);
  rpc search_property (SearchPropertyRequest) returns (SearchPropertyResponse);
  rpc book_property (BookPropertyRequest) returns (BookPropertyResponse);
  rpc confirm_booking (ConfirmBookingRequest) returns (ConfirmBookingResponse);
  rpc create_users (stream CreateUserRequest) returns (CreateUsersSummary);
  rpc list_available_properties (ListAvailableRequest) returns (stream Property);
}
```

Point at the two `stream` keywords — this is the highest-value thing to explain:

"The `stream` keyword on the **request** side of `create_users` makes it client-side streaming — the client sends many user profiles and the server replies once at the end. The `stream` on the **response** side of `list_available_properties` makes it server-side streaming — one request, and the server pushes properties back one at a time. The other six are simple unary calls. This single file is the contract; both the server and the client are generated from it, so they can never disagree."

Mention the generation command in the comment at the top of the file:

```
bal grpc --input proto/rental.proto --output server --mode service
```

4.3 Start the gRPC server

Go to **Terminal 3 (gRPC server)**:

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question2\server"
bal run
```

Expected:

```
Listening on : http://localhost:9090
Service : rental.RentalService
Properties : 6 seeded
Hosts / Guests : 3 / 3
RPCs exposed:
  add_property (unary)
  update_property (unary)
  remove_property (unary)
  search_property (unary)
```

```
book_property (unary)
confirm_booking (unary)
create_users (client streaming)
list_available_properties (server streaming)
Seeded host ids : HOST-001, HOST-002, HOST-003
Seeded guest ids: GUEST-001, GUEST-002, GUEST-003
```

"The server prints its own contract on start-up — all eight RPCs, and the seeded ids we'll use in the demo."

4.4 Run the gRPC client

Terminal 4 (gRPC client):

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question2\client"
bal run
```

Menu:

1. Create Users (CLIENT STREAMING)
2. Add Property (unary)
3. Update Property (unary)
4. Delete Property (unary)
5. List Available Properties (SERVER STREAMING)
6. Search Property (unary)
7. Book Property (unary)
8. Confirm Booking (unary)
9. Exit

The demo sequence — and what to say

Run in this order; it follows a real booking journey.

Step 1 — Press **1** (Create Users — client streaming)

Answer **y** to stream the built-in batch of four profiles.

You will see each profile sent, then a single summary:

```
-> sent [1] Frans Kapenda (HOST)
-> sent [2] Anna Nghifikwa (HOST)
-> sent [3] Lukas Tjitendero (GUEST)
-> sent [4] Rauha Shivute (GUEST)
Message : Received 4 profile(s): 2 host(s) and 2 guest(s) registered, 0 rejected.
```

"This is client-side streaming. The client opened one call, pushed four separate messages down it, then closed the stream — and only then did the server reply, with a single summary covering all four. That's exactly what the brief specified: multiple profiles streamed, one confirmation."

Step 2 — Press **2** (Add Property — unary)

- Host id: press **Enter** (defaults to **HOST-001**)
- Name: **Etosha Safari Cottage**
- Location: **Kunene**
- Type: press **Enter** for **APARTMENT**

- Price per night: 1500
- Maximum guests: 4
- Description: anything
- Amenities: WIFI, POOL

"A host registers a listing and the server returns a generated unique `property_id` — PROP-101. The client remembers it for the next steps."

Step 3 — Press 3 (Update Property — unary)

- Press **Enter** through the prompts until *New price per night*
- New price: 1600
- Press **Enter** for the rest

"The host updates the price using the `property_id` as the key. Any field left at zero or blank is left unchanged — a partial update."

Step 4 — Press 5 (List Available — server streaming)

Press **Enter** through all five filters to list everything.

"One request went out, and the server streamed the properties back one at a time — you can see them arrive row by row rather than as a single block. That's server-side streaming."

Step 5 — Press 5 again, this time with a filter

- Location contains: Swakopmund
- Press **Enter** for the rest

"The same streaming call with an optional filter applied. The brief asked for filtering by location or price range, and both are supported."

Optional extra: run it once with a location that doesn't exist, e.g. `Nowhere`. The server replies **"No property matched those filters"** rather than returning an empty stream.

Step 6 — Press 6 (Search Property — unary)

Press **Enter** to use the property you just created.

"A guest looks up one property by its id and gets the full details back. If the id doesn't exist, or the property isn't available, the server returns a 'Not Available' status instead — that's the requirement from the brief."

Step 7 — Press 7 (Book Property — the cart)

- Press **Enter** for property id and guest id
- Check-in: 2027-08-01
- Check-out: 2027-08-04

[OK] Request 'BKG-1001' added to your cart: 3 night(s) at Etosha Safari Cottage, estimated NAD 4800.0. Call `confirm_booking` to finalise it.

"Booking is deliberately two-phase. This first call validates the dates and puts the request in a temporary cart — it does **not** reserve the property yet."

Step 8 — Try a deliberately invalid booking

Press **7** again and enter a check-out date **before** the check-in, for example check-in `2027-09-10` and check-out `2027-09-05`:

```
[FAILED] 'checkOut' (2027-09-05) must be strictly after 'checkIn' (2027-09-10).
```

"That's the basic validation the brief asked for — the end date must be after the start date."

Step 9 — Press **8** (Confirm Booking)

Press **Enter** for both prompts.

```
[OK] Booking BKG-1001 confirmed: 3 night(s) at Etosha Safari Cottage,
    total NAD 4800.00.
Nights : 3
Total cost : NAD 4800.00
State : CONFIRMED
```

"Confirmation does three things. It re-checks that the property is still free for those dates so two guests can't double-book. It calculates the total — 1600 a night times 3 nights is 4800. And it clears the guest's temporary cart. Do the arithmetic with them: $1600 \times 3 = 4800$."

Step 10 — Press **4** (Delete Property)

- Property id: the one you created, e.g. `PROP-101`
- Host id: press **Enter**
- Confirm: **y**

"When a host removes a listing, the server responds with the updated list of available properties in that host's region — which is exactly the behaviour the brief specified for `remove_property`."

Step 11 — Press **9** to exit.

4.5 Tie it back to the mark sheet

"That's all eight operations: the protocol buffer contract, a client that can invoke every remote function including both streaming styles, and server-side logic handling storage, validation, overlap detection and price calculation."

Part 5 — Running the automated tests

This is a very strong way to close. It proves the system works rather than just showing it.

Stop the servers first (**Ctrl+C** in Terminals 1 and 3). The tests start their own listeners, and the Question 2 tests use port 9090 — the same port as the running gRPC server — so they will fail with *"Address already in use"* if you skip this.

Question 1 tests

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question1\service"  
bal test
```

Expected:

```
[pass] testAssetCrudAndFilters  
[pass] testComponentsAndOverdueFiltering  
[pass] testConcurrentLoanAttempts  
[pass] testFaultReportedWhileOnLoan  
[pass] testInstitutionRegistrationAndWithdrawal  
[pass] testLoanReturnAndStatusProtection  
[pass] testNestedPayloadValidation  
[pass] testScheduleModificationAndBookingConflicts  
[pass] testWorkOrderTaskLifecycle
```

```
9 passing  
0 failing
```

Call out one test by name:

"testConcurrentLoanAttempts fires thirty-two simultaneous loan requests at the same asset and asserts that exactly one succeeds. That's the concurrency guarantee, proven rather than claimed."

Question 2 tests

```
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question2\server"  
bal test
```

Expected:

```
[pass] testAllRentalOperations  
[pass] testConcurrentUpdatesAndRegistration  
[pass] testCalendarValidation
```

```
3 passing  
0 failing
```

"testAllRentalOperations exercises all eight RPCs end to end, including two guests racing to confirm the same dates — only one gets it."

Note: `Question1/service/tests/Config.toml` puts the test listener on port 8085 so the tests run even while XAMPP holds 8080.

Part 6 — Troubleshooting during the live demo

Keep this page open on a phone or second screen.

Symptom	Cause	Fix
<code>bal: command not found</code>	Ballerina not on PATH	Use the full path " <code>C:\Program Files\Ballerina\bin\bal.bat</code> ", or fix PATH (§0.1) and open a new terminal
<code>Address already in use: bind</code> on 8080	XAMPP Apache has the port	Start with <code>-CservicePort=8081</code>
<code>Address already in use: bind</code> on 8081/9090	An older run is still alive	See "kill a stuck server" below
Client says <i>Cannot reach the API</i>	Server not running, or wrong port	Check Terminal 1; make sure client uses <code>-CapiUrl=http://localhost:8081</code>
Dashboard stuck on Connecting...	REST server down, or wrong <code>apiBase</code>	Confirm <code>curl http://localhost:8081/health</code> , and that the URL has <code>?apiBase=http://localhost:8081</code>
Dashboard shows nothing and console shows a CORS error	You opened the <code>.html</code> file directly	Serve it over HTTP — §3.1
Weird 500s / timeouts after you rebuilt	Server is running an old build	<code>Ctrl+C</code> , <code>bal build</code> , start again
Compilation takes ages mid-demo	Not pre-built	Always run §0.3 beforehand

Kill a stuck server

```
netstat -ano | findstr ":8081"
```

Take the number in the last column (the PID) and:

```
taskkill /PID <number> /F
```

The universal reset

If anything goes badly wrong, calmly do this:

1. `Ctrl+C` in every terminal.
2. Close all terminals.
3. Reopen and start the server you need.

Say: *"Let me restart that cleanly"* — and carry on. Staying composed matters more than the glitch.

Part 7 — Questions the lecturer is likely to ask

"Why Ballerina rather than Java or Node?"

Ballerina treats network calls as first-class language constructs. A REST resource and a gRPC remote function are language syntax, not framework annotations, so there is far less boilerplate between the design and the running service. It also has `isolated` and `lock` built into the type system for safe concurrent state.

"Where is your database?"

The brief specified a Map or a table. We used a Ballerina `table` keyed on `assetTag` for Question 1 and maps for Question 2, both held in memory and guarded by locks. Data resets when the service restarts — a real deployment would swap the `database.bal` layer for a real database, and nothing above it would change, because the service layer only talks to those functions.

"What happens if two people loan the same asset at once?"

Exactly one succeeds. The availability check and the status change happen inside a single lock, so the second request sees the updated status and is rejected with HTTP 409. We have a test that fires 32 concurrent requests and asserts one winner.

"Why is booking split into two calls?"

The brief asked for a temporary cart. It also mirrors how real booking works — the price and availability are only fixed at confirmation. It also keeps the window in which we hold a reservation as short as possible.

"How do you prevent double bookings on overlapping dates?"

`confirm_booking` re-checks the dates against every confirmed booking for that property before it commits, inside a lock. Two guests confirming the same dates at the same time result in exactly one confirmation — that's covered by a test.

"When would you choose REST over gRPC?"

REST when the consumer is a browser or a third party who wants something readable and cacheable over plain HTTP — which is why the web dashboard talks to Question 1. gRPC when both ends are services you control and you want a strict contract, smaller binary messages and native streaming.

"How is the total cost calculated?"

Price per night multiplied by the number of nights, where nights is the difference between check-in and check-out. The date maths handles month ends and leap years — there's a test for 29 February.

"What would you do next?"

Persist to a real database, add authentication so only the real host can edit a listing, and add pagination to the listing endpoints.

Part 8 — Timing plan and speaking roles

Suggested 20-minute plan

Group 22, eight members — roughly two and a half minutes each.

Time	Section	Who
0:00–1:00	Title, team, the two problems	Matatias Nghihangwa
1:00–3:00	Why Ballerina; repository layout	Eliaser Angula (<i>leader</i>)
3:00–5:30	Q1 architecture, domain model, storage	Pandera Katjipuka
5:30–7:30	Q1 error handling and endpoint tour	Monika Shalauda
7:30–11:00	Q1 live demo, then the web dashboard	Risto Sakeus
11:00–13:00	Q2 <code>.proto</code> contract; streaming explained	Alanray Miller
13:00–15:00	Booking rules and concurrency	Kavara Edward
15:00–18:00	Q2 live demo; REST vs gRPC; next steps	Haufiku Frans
18:00–20:00	Questions	Everyone

This matches the speaking-role split in `[PRESENTATION.md](PRESENTATION.md)` — keep the two in step if you change it.

Rules for the team

- **Everyone speaks.** The brief says all members must contribute and groups must defend the solution.
 - Whoever is not speaking watches the terminals — if a server dies, restart it quietly rather than interrupting.
 - Know *one* part deeply each, but be able to answer basics on all of it.
 - If you don't know an answer: *"We didn't implement that, but the way we'd approach it is..."* — far better than guessing.
-

Part 9 — One-page command cheat sheet

Print this page or keep it on a second screen.

```
# ---- Setup (do before presenting) -----
bal version # expect 2201.13.5
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment"
.\scripts\run.ps1 -Target build # build all four packages

# ---- Question 1 — REST -----
# Terminal 1 — server
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question1\service"
bal run -- -CservicePort=8081

# Terminal 2 — client
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question1\client"
bal run -- -CapiUrl=http://localhost:8081

# quick check
curl http://localhost:8081/health

# ---- Web dashboard -----
# Terminal 5
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment"
.\scripts\run.ps1 -Target web
# then browse to:
# http://localhost:5500/?apiBase=http://localhost:8081

# ---- Question 2 — gRPC -----
# Terminal 3 — server
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question2\server"
bal run

# Terminal 4 — client
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question2\client"
bal run

# ---- Tests (stop the servers first) -----
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question1\service"
bal test
cd "C:\xampp\htdocs\DISTRIBUTIVE SYSTEM\DSA612S-Assignment\Question2\server"
bal test

# ---- Free a stuck port -----
netstat -ano | findstr ":8081"
taskkill /PID <number> /F
```

Demo data you will need

Question 1 — seeded assets

Asset tag	Name	Institution
NUST-LIB-3DP-001	Pro-Series 3D Printer	NUST

Asset tag	Name	Institution
NUST-LIB-LAP-014	Dell Latitude 5540 Loan Laptop	NUST
NUST-LIB-ROOM-002	Postgraduate Discussion Room 2	NUST
UNAM-LIB-TC-207	HP t640 Thin Client	UNAM
UNAM-LIB-BOOK-8891	Distributed Systems (5th Ed.)	UNAM
IUM-LIB-PRN-033	Konica Minolta bizhub C300i	IUM

Question 2 — seeded data

Property	Name	Location
PROP-001	Atlantic Dune Apartment	Swakopmund
PROP-002	Mole Beach Cottage	Swakopmund
PROP-003	Klein Windhoek Guesthouse	Windhoek
PROP-004	Auas Hills Villa	Windhoek
PROP-005	Okaukuejo Safari Chalet	Etosha
PROP-006	Namutoni Rest Camp Room	Etosha

Hosts: HOST-001, HOST-002, HOST-003 Guests: GUEST-001, GUEST-002, GUEST-003

End of runbook.